DESIGN OF SYNCHRONOUS SEQUENTIAL LOGIC

LAB 13

B113040045 許育菖

2023/11/27

EXERCISE 1

DESIGN OF SEQUENCE DETECTOR (D-FF)

題目說明

Design a sequence detector to detect a sequence of three or more consecutive 1’s in a string of bits coming through an input line

- The state diagram and state table of the sequence detector are shown in **Fig. 5.27** and **Table 5.11**
- Design the sequence detector with D flip-flops
- Write the Verilog HDL description of the state diagram (i.e., **Behavioral model**)
- Write the Verilog HDL description of the logic circuit diagram (i.e., a **Structural model**)
- Write an Verilog HDL stimulus with a sequence of inputs: 0111110110. Verify that the responses are the same for both descriptions (first reset all flip-flops to 0).

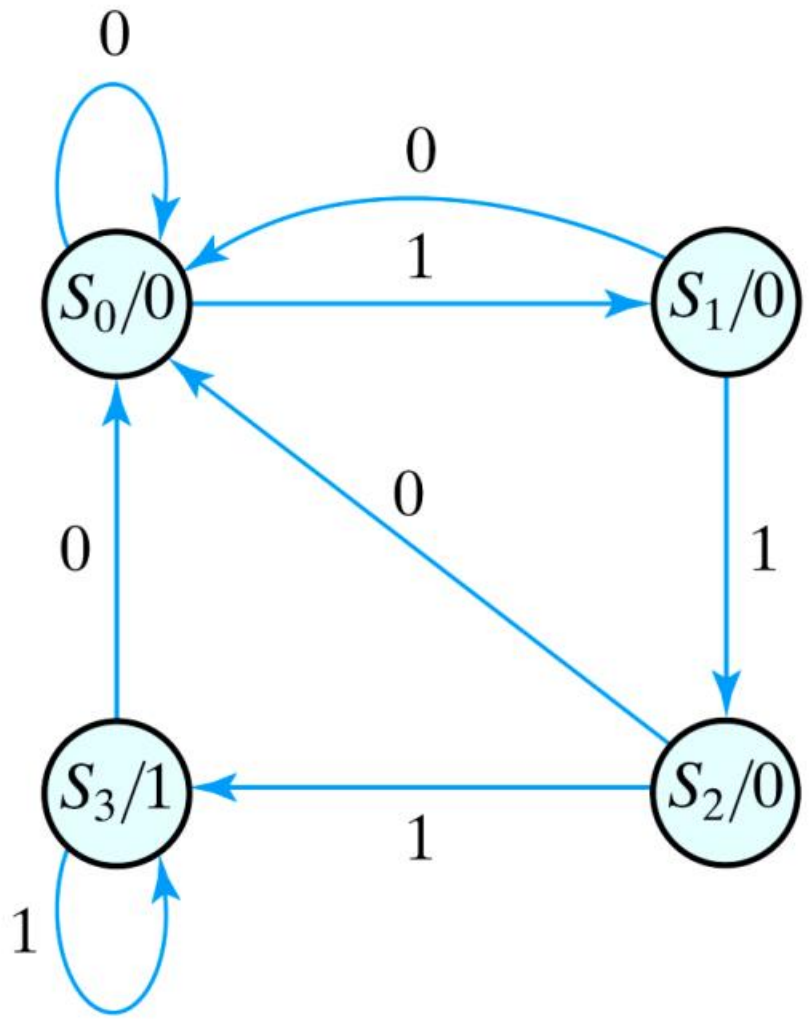


Table 5.11
State Table for Sequence Detector

Present State		Input	Next State		Output
A	***B***	***x***	***A***	***B***	***y***
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

Fig. 5.27 State diagram for sequence detector

VERILOG CODE

state diagram

依照Fig 5.27的狀態圖，根據當前的state以及x的值，得出下一個state

```
`timescale 1ns / 1ps
module exercise1_state(
    output y,
    input  x, clock, reset );

    reg [1:0] state;
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;

always @ (posedge clock, negedge reset)
  if(reset == 0)
    state <= S0;
  else
    case(state)
      S0: if(x == 1) state <= S1;
        else       state <= S0;
      S1: if(x == 1) state <= S2;
        else       state <= S0;

      S2: if(x == 1) state <= S3;
        else       state <= S0;
      S3: if(x == 0) state <= S0;
        else       state <= S3;
    endcase
  assign y = (state == S3);
endmodule
```

VERILOG CODE

logic circuit diagram

D_A

A \ Bx	00	01	11	10
0	0	0	1	0
1	0	1	1	0

D_B

A \ Bx	00	01	11	10
0	0	1	0	0
1	0	1	1	0

首先把D flip-flop的邏輯電路寫好。

接著根據**Table 5.11**，找出表中next state A、next state B值為1的列，我們可以得到：

$$D_A = A'Bx + AB'x + ABx$$

$$D_B = A'B'x + AB'x + ABx$$

畫出K-map並化簡後求出：

$$D_A = Ax + Bx$$

$$D_B = B'x + Ax$$

並且由**Table 5.11**，可以看出：

$$y = AB$$

由以上三個算式撰寫出verilog logic circuit diagram

VERILOG CODE

logic circuit diagram

```
// D flip_flop
`timescale 1ns / 1ps

module D_flip_flop(
    output reg Q,
    input D, clk, rst);

    always @ (posedge clk, negedge rst)
        if(rst == 0)
            Q <= 1'b0;
        else
            Q <= D;
endmodule
```

```
// logic circuit
`timescale 1ns / 1ps
module exercise1(y, x, clock, reset);
    output y;
    input x, clock, reset;

    wire DA, DB, QA, QB;

    assign DB = ( QA & x ) | ( (!QB) & x );
    assign DA = ( QA & x ) | ( QB & x );
    assign y = QB & QA;

    D_flip_flop out2(QB, DB, clock, reset);
    D_flip_flop out1(QA, DA, clock, reset);

endmodule
```

TESTBENCH

分成兩個部分。

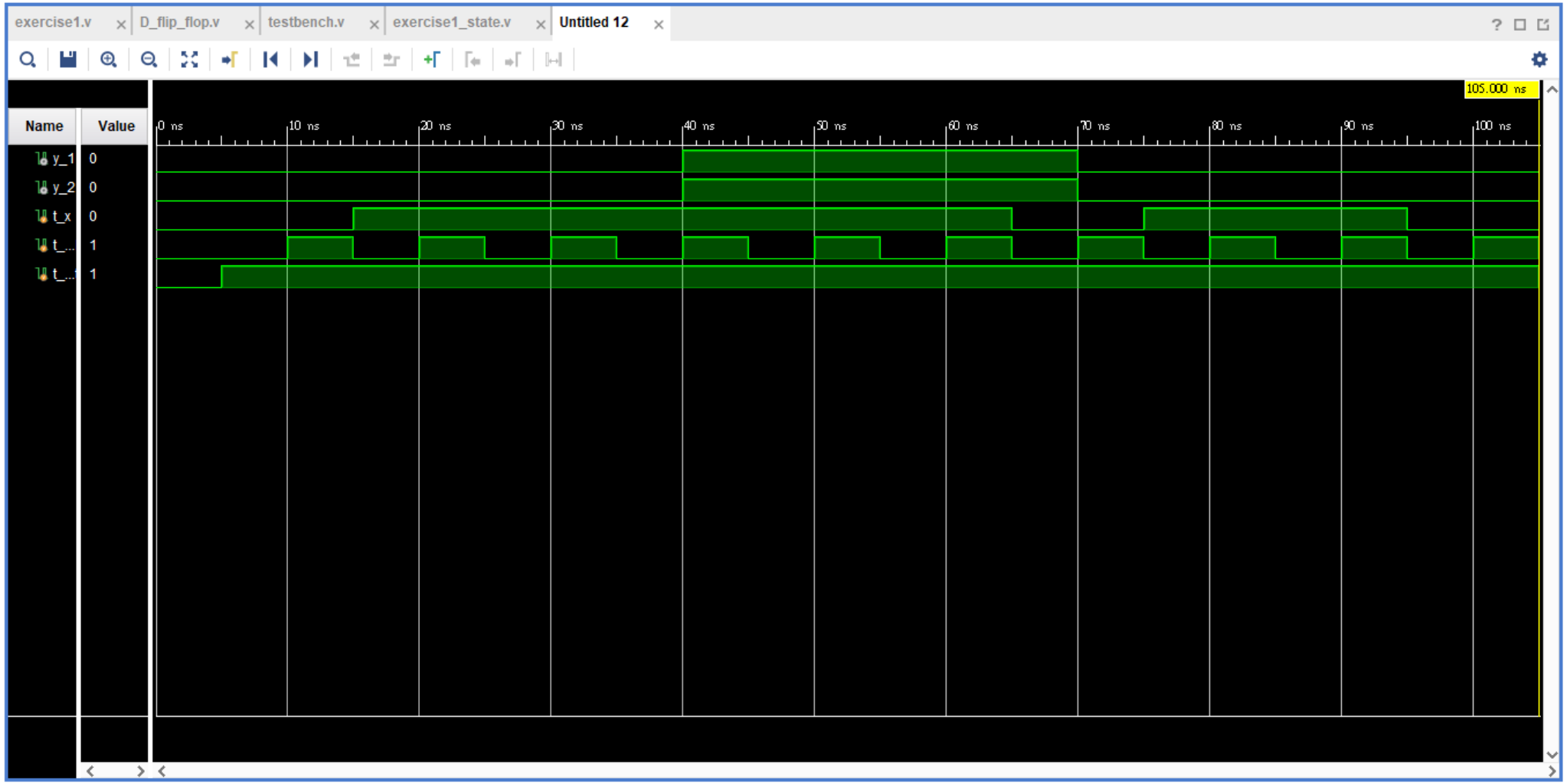
第一部分處理clock和reset的變化；

第二部分按題目要求，依序輸入數列

0111110110

```
`timescale 1ns / 1ps
module testbench();
    wire y_1, y_2;
    reg t_x;
    reg t_clock, t_reset;

    exercise1 UO (y_1, t_x, t_clock, t_reset);
    exercise1_state U1 (y_2, t_x, t_clock, t_reset);

    initial #105 $finish;
    initial begin
        t_reset = 0;
        t_clock = 0;
    #5  t_reset = 1;
    repeat(20)
    #5  t_clock = ~t_clock;
    end
initial begin
        t_x = 1'b0; #15;
        t_x = 1'b1; #10;
        t_x = 1'b1; #10;
        t_x = 1'b1; #10;
        t_x = 1'b1; #10;

        t_x = 1'b1; #10;
        t_x = 1'b0; #10;
        t_x = 1'b1; #10;
        t_x = 1'b1; #10;
        t_x = 1'b0; #10;
    end
endmodule
```

實驗結果


exercise1.v
D_flip_flop.v
testbench.v
exercise1_state.v
Untitled 12
105.000 ns
Name
Value
y_1 0
y_2 0
t_x 0
t_... 1
t_... 1
0 ns
10 ns
20 ns
30 ns
40 ns
50 ns
60 ns
70 ns
80 ns
90 ns
100 ns

EXERCISE 2

DESIGN OF SEQUENCE DETECTOR (T-FF)

題目說明

Design a sequence detector to detect a sequence of three or more consecutive 1’s in a string of bits coming through an input line

- The state diagram and state table of the sequence detector are shown in **Fig. 5.27** and **Table 5.11**
- Design the sequence detector with T flip-flops
- Write the Verilog HDL description of the state diagram (i.e., **Behavioral model**)
- Write the Verilog HDL description of the logic circuit diagram (i.e., a **Structural model**)
- Write an Verilog HDL stimulus with a sequence of inputs: 0111110110. Verify that the responses are the same for both descriptions (first reset all flip-flops to 0).

VERILOG CODE

state diagram

依照Fig 5.27的狀態圖，根據當前的state以及x的值，得出下一個state

（程式碼與第一題基本相同）

```
`timescale 1ns / 1ps
module exercise2_state(
    output y,
    input  x, clock, reset );

    reg [1:0] state;
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
```

```
always @ (posedge clock, negedge reset)
  if(reset == 0)
    state <= S0;
  else
    case(state)
      S0: if(x == 1) state <= S1;
          else       state <= S0;
      S1: if(x == 1) state <= S2;
          else       state <= S0;

      S2: if(x == 1) state <= S3;
          else       state <= S0;
      S3: if(x == 0) state <= S0;
          else       state <= S3;
    endcase
  assign y = (state == S3);
endmodule
```

VERILOG CODE

logic circuit diagram

首先於**Table 5.11**新增T_A、T_B的值：

$$T_A = A_{present} \oplus A_{next}$$

$$T_B = B_{present} \oplus B_{next}$$

Present State		Input	Next State	
A	***B***	***x***	***A***	***B***
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	0	0
1	1	1	1	1

T_A	T_B
0	0
0	1
0	1
1	1
1	0
0	1
1	1
0	0

VERILOG CODE

logic circuit diagram

T_A

A \ Bx	00	01	11	10
0	0	0	1	0
1	1	0	0	1

T_B

A \ Bx	00	01	11	10
0	0	1	1	1
1	0	1	0	1

接著根據更改過的**Table 5.11**，找出表中next state A、next state B值為1的列，我們可以得到：

$$T_A = A'Bx + AB'x' + ABx'$$

$$T_B = A'B'x + A'Bx' + A'Bx + AB'x + ABx'$$

畫出K-map並化簡後求出：

$$T_A = Ax' + A'Bx$$

$$T_B = B'x + A'x + Bx'$$

並且由**Table 5.11**，可以看出：

$$y = AB$$

由以上三個算式，加上額外撰寫的T flip-flop，完成verilog logic circuit diagram

VERILOG CODE

logic circuit diagram

```
//        T flip-flop
`timescale 1ns / 1ps
module T_flip_flop(
    output reg Q,
    input T, clk, rst
    );

    always @ (posedge clk, negedge rst)
        if(rst == 0)
            Q <= 1'b0;
        else
            Q <= (T^Q);
endmodule
```

```
//        logic circuit
`timescale 1ns / 1ps
module exercise_structure(y, x, clock, reset);
    output y;
    input x, clock, reset;
    wire TA, TB, QA, QB;

    assign TB = ( (!QB) & x ) | (  QB & (!x) ) | ( (!QA) & x );
    assign TA = ( QA & (!x) ) | ( (!QA) & QB & x );
    assign y = QB & QA;

    T_flip_flop out2(QB, TB, clock, reset);
    T_flip_flop out1(QA, TA, clock, reset);

endmodule
```

TESTBENCH

分成兩個部分。

第一部分處理clock和reset的變化；

第二部分按題目要求，依序輸入數列

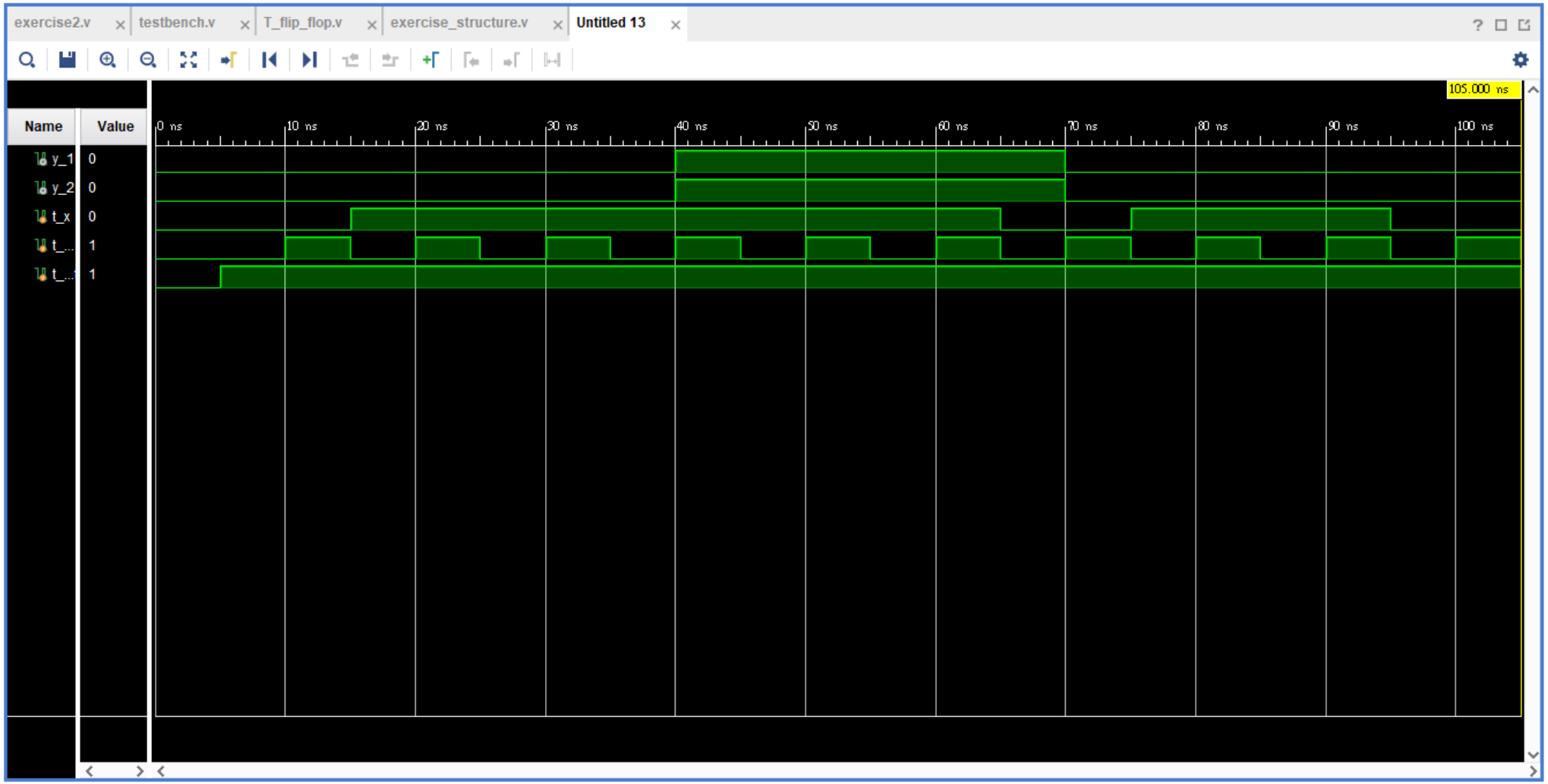
0111110110

（程式碼與第一題基本相同）

```
`timescale 1ns / 1ps
module testbench();
    wire y_1, y_2;
    reg t_x;
    reg t_clock, t_reset;

    exercise_structure UO (y_1, t_x, t_clock, t_reset);
    exercise2 U1 (y_2, t_x, t_clock, t_reset);

    initial #105 $finish;
    initial begin
        t_reset = 0;
        t_clock = 0;
    #5  t_reset = 1;
    repeat(20)
    #5  t_clock = ~t_clock;
    end
```

```
initial begin
        t_x = 1'b0; #15;
        t_x = 1'b1; #10;
        t_x = 1'b1; #10;
        t_x = 1'b1; #10;
        t_x = 1'b1; #10;

        t_x = 1'b1; #10;
        t_x = 1'b0; #10;
        t_x = 1'b1; #10;
        t_x = 1'b1; #10;
        t_x = 1'b0; #10;
    end
endmodule
```

實驗結果


exercise2.v
testbench.v
T_flip_flop.v
exercise_structure.v
Untitled 13
105.000 ns
Name
Value
y_1 0
y_2 0
t_x 0
t_... 1
t_... 1
0 ns
10 ns
20 ns
30 ns
40 ns
50 ns
60 ns
70 ns
80 ns
90 ns
100 ns

實驗心得

MY REFLECTION

此次實驗練習了D flip-flop和T flip-flop的sequence detector。

我認為這次實驗在沒搞清楚「怎麼從真值表找出公式並帶入K-map化簡？」時，會非常得難寫，甚至完全沒有頭緒程式碼要如何下手，這就是我一開始遇到的情況。我自己是最後在打這份實驗結報時才完全搞懂sequence detector。

該問題的解決方法為：

1. 首先從真值表中找出所有flip-flop輸入為1的列，並把這些present state和input狀態的積加起來，得出flip-flop輸入值的公式
2. 接著畫出該公式的K-map並化簡
3. 最後以得到的最簡公式撰寫出**行為描述模型**

MY REFLECTION